FPT UNIVERSITY
SOFTWARE ENGINEER MAJOR
LAB211

FPT UNIVERSITY

HOMESTAY BOOKING MANAGEMENT SYSTEM REPORT

SUPERVISOR : Ms. Than Thi Ngoc Van
SUBJECT : LAB211
CLASS : SE2043

STUDENT : Nguyen Van Gia Binh (SE203555)

HO CHI MINH CITY 3/2026

Contents

Chapter 1 Introduction 2
1.1 Project Overview 2
1.2 Scope and Target Audience 2
Chapter 2 System Overview 3
Chapter 3 4
System Architecture 4
3.1 Layered Architecture 4
3.2 Class Diagram 4
Chapter 4 6
Software Flow and Pseudocode 6
4.1 Function 1: Add a new Tour 6
4.1.1 Function Workflow Description 8
4.2 Function 2: Update Tour by ID 8
4.2.1 Update Function Workflow 11
4.3 Function 3: List Tours Earlier Than Current Date 12
4.3.1 Earlier Tours Function Workflow 12
4.4 Function 4: List Total Booking Amount Later Than Current Date 13
4.4.1 List Upcoming Tours Workflow 14
4.5 Function 5: Create a new booking 14
4.5.1 Booking Creation Workflow 16
4.6 Function 6: Remove booking 16
4.6.1 Booking Cancelation Workflow 17
4.7 Function 7: Update booking 18
4.7.1 Booking Modification Workflow 19
4.8 Function 8: Search Booking 20
4.8.1 Search Booking Workflow 20
4.9 Function 9: StatisticTotalTourists 21
4.9.1 Statistics Workflow 22
Chapter 6 23
6.1 Business Logic Architecture 23
6.1.1 Tour Management Logic 23
6.1.2 Booking Management Logic 23
6.1.3 Reporting and Analytics 24
Chapter 7 Conclusion 25

Chapter 1

Introduction

1.1 Project Overview

The Homestay Booking Management Application is a console-based Java application. The application addresses the need for a structured system to manage home stay accommodations, tour packages, and booking records.

In Vietnam's rapidly growing tourism market, home stay owners and travel operators need efficient tools to manage tour information, track bookings, and generate statistical reports. This application provides a simplified yet functional solution for managing these core operations through a text-based menu interface..

1.2 Scope and Target Audience

The primary objective of this project is to build a fully functional Homestay Booking Management System that allows **manager** to:

- **Manage Tours**: Add new tours, update tour information, and query tours based on date criteria.
- **Manage Bookings**: Create, update, remove, and search booking records.
- **Generate Reports**: view statistical summaries such as total booking amounts and tourist counts per home stay.
- **Persist Data**: Save and load all data from text files to ensure data persistence between sessions.

Chapter 2

System Overview

The system operates as a menu-driven application with 10 main functions:

1. **Add new tour:** Create the new tour with validation (tour's informations, homestay booking, validation between dates).
2. **Update a tour ID:** Modify the attributes of an existing tour.
3. **List the Tours with departure dates earlier than the current date:** Find tours with departure dates earlier than today.
4. **List total Booking amount for future tours:** List the total booking amount for tours with departure dates later than the current date.
5. **Add a new Booking:** Create a new booking.
6. **Remove a Booking by bookingID:** Delete an existing booking.
7. **Update a Booking by bookingID:** Modify an existing booking.
8. **List all Booking by fullName:** Find and display the booking information by full or partial name.
9. **Statistics:** Statistics on the total number of tourists who have booked each homestay.
10. **Quit program:** Upon exit, if any related data files have been modified, the system shall prompt the user to save changes.

Chapter 3

System Architecture

3.1 Layered Architecture

View: Main, Menu – Display the console menu and interact with the user.

Controller: Controller – Handle user choices, connect View to Service layer.

Service: TourList, HomestayList, BookingList, IService – Include business logic, validation, and data processing for each entity.

Model: Tour, Homestay, Booking – Define the data structure for objects.

File Utilities: FileUtils – Read/write data to files.

Utilities: Inputter, Validator – Provide input handling and validation tools for all layers.

3.2 Class Diagram

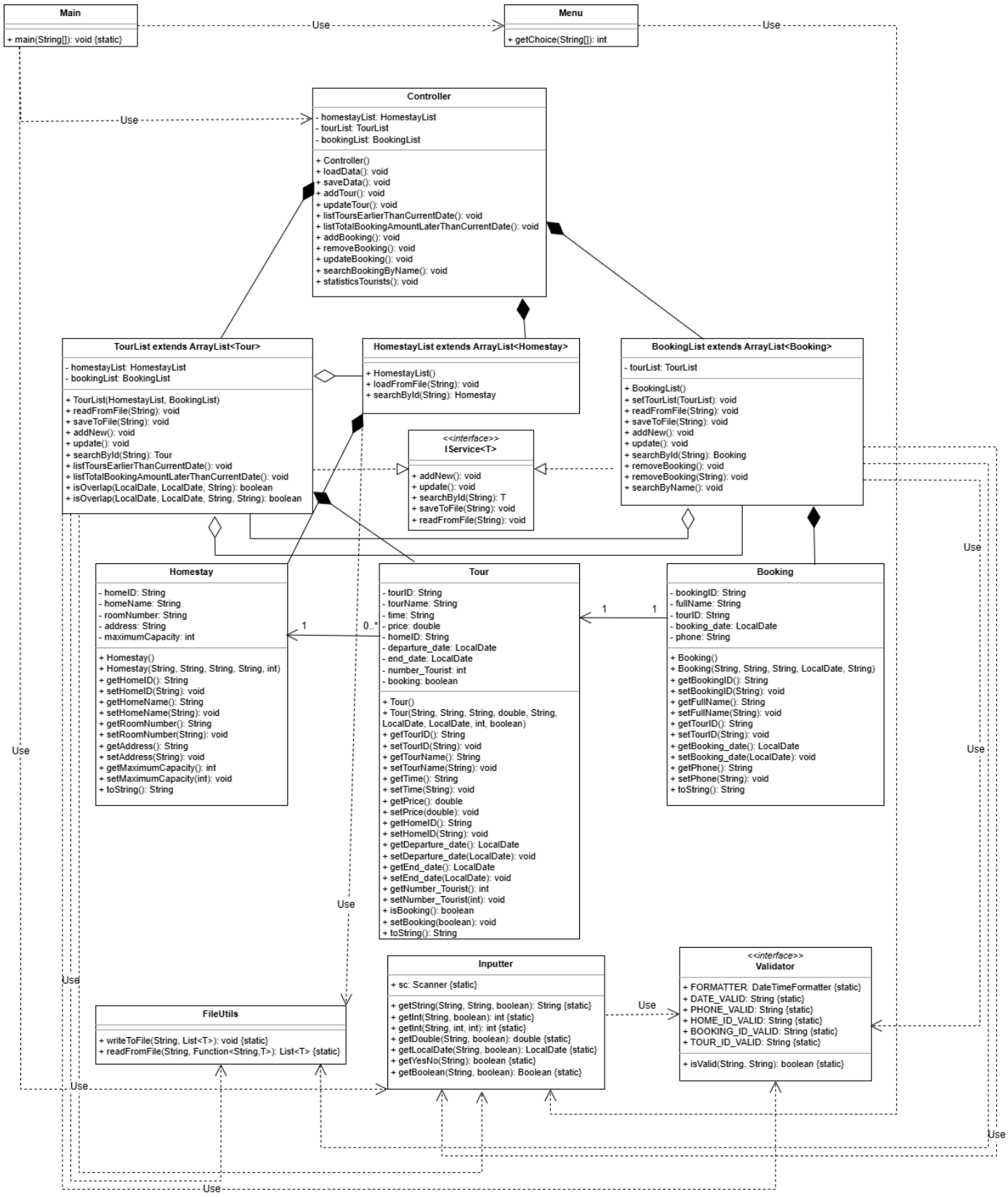

Figure 3.1: Detailed Class Diagram of the System

Chapter 4

Software Flow and Pseudocode

4.1 Function 1: Add a new Tour

```
ALGORITHM  Add New Tour BEGIN

   PRINT "--- Add new Tour ---"

   REPEAT
      INPUT tourID (format: TXXXXX)
      Search tour record by tourID

      IF tour exists THEN
          PRINT "Error: Tour ID already exists!"
       ELSE
          BREAK LOOP
       END IF
    END REPEAT

    INPUT tourName

    INPUT time (String)

    INPUT price (double)

    REPEAT
       INPUT homeID (format: HSXXXX)
       Find homestay record by homeID

       IF homestay is NOT found THEN
          PRINT "Error: Homestay not found in the system!"
       ELSE
          BREAK LOOP
```

```
        END IF
    END REPEAT

    REPEAT
        INPUT departureDate (format: dd/MM/yyyy)
        INPUT endDate (format: dd/MM/yyyy)

        IF endDate is before departureDate THEN
            PRINT "Error: End date must be >= departure date."
        ELSE IF departureDate is before today THEN
            PRINT "Error: New tour can not start in the past."
        ELSE IF schedule has overlap with another tour in the same homestay THEN
            PRINT "Error: Tour date overlaps with another tour in the same homestay."
        ELSE
            BREAK LOOP
        END IF
    END REPEAT

    Get maximumCapacity of the chosen homestay

    REPEAT
        INPUT numTourists

        IF numTourists >= maximumCapacity THEN
            PRINT "Error: Number of tourists must be less than maximum capacity (" + maximumCapacity + ")."
        ELSE
            BREAK LOOP
        END IF
    END REPEAT

    Create a new tour record with (tourID, tourName, time, price, homeID, departureDate, endDate, numTourists, booking = false)
    Add the new tour to the tour list

    PRINT "[Success] Tour: " + tour details + " has been added successfully."

END
```

Listing 4.1: Add tour pseudocode

4.1.1 Function Workflow Description

The following steps describe the logical flow of the addTour function:

Step	Description
Step 1: Identity Validation	The system prompts the user to input a Tour ID following the format TXXXXX. The input is validated using a Regex pattern. The system then checks if the entered ID already exists by calling searchById(tourID). If a duplicate is found, an error message is displayed. This loop repeats until a unique, valid Tour ID is provided.
Step 2: Basic Information Input	The user enters the Tour Name, the Time/Duration (e.g., "6 days 5 nights"), and the Price. Each input is collected using the Inputter utility class, which handles format validation.
Step 3: Homestay Association	The system requests a Home ID following the format HSXXXX. The system queries homestayList.searchById(homeID) to verify that the homestay exists. If not found, an error is printed and the user must re-enter.
Step 4: Schedule Logic and Conflict Check	The user inputs the Departure Date and End Date in dd/MM/yyyy format. The system checks: (1) End Date must not be before Departure Date, (2) Departure Date must not be in the past, (3) No overlap with other tours using the same homestay via isOverlap().
Step 5: Capacity Verification and Persistence	The user specifies the Number of Tourists. The system validates it is less than the homestay's maximumCapacity. A new Tour object is created with booking = false and added to the list. A success message is printed.

4.2 Function 2: Update Tour by ID

```
ALGORITHM Update Tour By ID BEGIN

  PRINT "--- Update Tour ---"
  SET today = current date

  REPEAT
    INPUT tourID (format: TXXXXX)
    Search tour record by tourID

     IF tour is NOT found THEN
        PRINT "Error: This tour does not exist!"
     ELSE IF tour's departureDate is before today THEN
        PRINT "Error: Can not update tour has already started."
     ELSE
        BREAK LOOP
     END IF
  END REPEAT

  PRINT "Current Name: " + tour.tourName
  INPUT newTourName (Enter to skip)
  IF newTourName is NOT empty THEN
     SET tour.tourName = newTourName
  END IF

```

```
PRINT "Current Time: " + tour.time
INPUT newTime (Enter to skip)
IF newTime is NOT empty THEN
   SET tour.time = newTime
END IF

PRINT "Current Price: " + tour.price
INPUT newPrice (Enter or 0 to skip)
IF newPrice > 0 THEN
   SET tour.price = newPrice
END IF

REPEAT
   PRINT "Current Home ID: " + tour.homeID
   INPUT newHomeID (Enter to skip)

   IF newHomeID is empty THEN
      BREAK LOOP
   END IF

   IF homestay is NOT found by newHomeID THEN
      PRINT "Error: Homestay not found."
   ELSE
      SET tour.homeID = newHomeID
      BREAK LOOP
   END IF
END REPEAT

REPEAT
   INPUT newDepartureDate (Enter to skip)
   INPUT newEndDate (Enter to skip)

   IF newDepartureDate is null THEN SET newDepartureDate = tour.departureDate
   IF newEndDate is null THEN SET newEndDate = tour.endDate

   IF newDepartureDate is after newEndDate THEN
      PRINT "Error: End date must be >= departure date."
   ELSE IF newDepartureDate is before today THEN
      PRINT "Error: New tour can not start in the past."
   ELSE IF schedule has overlap with another tour (excluding current tour) THEN
      PRINT "Error: Tour date overlaps with another tour in the same homestay."
   ELSE
      SET tour.departureDate = newDepartureDate
      SET tour.endDate = newEndDate
      BREAK LOOP
   END IF
END REPEAT

REPEAT
   PRINT "Current Number of Tourists: " + tour.numTourists
   INPUT newNumTourists (Enter or 0 to skip)

   IF newNumTourists == 0 THEN
```

```
            BREAK LOOP
        END IF

        Get maximumCapacity of the tour's homestay

        IF newNumTourists >= maximumCapacity THEN
            PRINT "Error: Number of tourists must be less than maximum capacity (" + maximumCapacity + ")."
        ELSE
            SET tour.numTourists = newNumTourists
            BREAK LOOP
        END IF
    END REPEAT

    INPUT newBookingStatus (Enter to skip)
    IF newBookingStatus is NOT null THEN
        IF newBookingStatus is false THEN
            Remove all bookings associated with this tourID from bookingList
        END IF
        SET tour.booking = newBookingStatus
    END IF

    PRINT "[Success] Tour: " + tour details + " has been updated!"

END
```

Listing 4.2: Pseudocode for update tour by tourID

4.2.1 Update Function Workflow

Step	Description
Step 1: Tour Identification & Validation	The system prompts the user to input a Tour ID in `TXXXXX` format. It calls searchById(tourID) to find the tour. If the tour does not exist, an error is shown. If the tour's departure date is before today, it cannot be updated (the tour has already started). The loop repeats until a valid, updatable tour is found.
Step 2: Update Basic Information	The system displays the current Tour Name, Time, and Price. For each field, the user can enter a new value or press Enter to skip. If a non-empty value is entered for Name or Time, or a value greater than 0 for Price, the corresponding field is updated.
Step 3: Update Homestay Association	The system displays the current Home ID and prompts for a new one. If the user presses Enter, the field is skipped. Otherwise, the system queries `homestayList.searchById(newHomeID)` to verify the homestay exists. If not found, an error is shown and the user must retry.
Step 4: Update Schedule with Conflict Check	The user inputs a new Departure Date and End Date (or presses Enter to keep the current values). The system validates: (1) End Date must not be before Departure Date, (2) Departure Date must not be in the past, (3) No overlap with other tours at the same homestay (excluding the current tour) via isOverlap().
Step 5: Update Tourist Capacity	The system displays the current Number of Tourists. The user enters a new value or 0 to skip. If entered, the value is validated against the homestay's `maximumCapacity` to ensure it is strictly less.
Step 6: Update Booking Status & Cascade Delete	The user can optionally update the booking status. If the new status is set to `false`, the system iterates through the `bookingList` and **removes all bookings** associated with this Tour ID. The tour's booking status is then updated accordingly. A success message is printed with the updated tour details.

4.3 Function 3: List Tours Earlier Than Current Date

```
ALGORITHM List Tours Earlier Than Current Date BEGIN

    PRINT "--- Tours with departure dates earlier than today ---"
    SET today = current date
    SET found = false

    PRINT table header "| TourID | Name | Departure |"

    FOR EACH tour IN tourList DO
        IF tour.departureDate is before today THEN
            PRINT tour.tourID, tour.tourName, tour.departureDate (formatted)
            SET found = true
        END IF
    END FOR

    IF found is false THEN
        PRINT "No tours found."
    END IF

END
```

Listing 4.3: List Tours Earlier Than Current Date

4.3.1 Earlier Tours Function Workflow

Step	Description
Step 1: Initialization	The system prints the section header and sets today's date using `LocalDate.now()`. A boolean flag `found` is initialized to `false` to track whether any matching tour exists. A table header row is printed to format the output.
Step 2: Iterate and Filter	The system iterates through every Tour object in the tour list. For each tour, it compares `tour.departureDate` with today using `isBefore(today)`. If the departure date is earlier than today, the tour's ID, name, and formatted departure date are printed in a table row using `printf`, and the `found` flag is set to `true`.
Step 3: Handle Empty Result	After the loop completes, the system checks the `found` flag. If no tour matched the condition (i.e., `found` is still `false`), it prints `"No tours found."` to inform the user that there are no past tours in the system.

4.4 Function 4: List Total Booking Amount Later Than Current Date

```
ALGORITHM List Total Booking Amount Later Than Current Date BEGIN

  PRINT "--- Tours with departure > today (Sorted by Total Amount Descending) ---"
  SET today = current date

  CREATE empty list validTours

  FOR EACH tour IN tourList DO
     IF tour.departureDate is after today AND tour.booking is true THEN
         ADD tour TO validTours
     END IF
  END FOR

  IF validTours is empty THEN
     PRINT "No tours found."
     RETURN
  END IF

  SORT validTours BY (price * numTourists) IN DESCENDING ORDER

  FOR EACH tour IN validTours DO
     SET totalAmount = tour.price * tour.numTourists
     PRINT tour.tourID, tour.departureDate, tour.price, tour.numTourists, totalAmount
  END FOR

END
```

Listing 4.4: List Total Booking Amount Later Than Current Date

4.4.1 List Upcoming Tours Workflow

Step		Description
Step 1: Initialization		The system prints the section header and retrieves today's date using `LocalDate.now()`. An empty list `validTours` is created to store tours that meet the filter criteria.
Step 2: Filter Valid Tours		The system iterates through every Tour in the tour list. A tour is added to `validTours` only if **both** conditions are met: (1) the departure date is **after** today, and (2) the tour's `booking` status is `true`. Tours that have already departed or are not booked are excluded.
Step 3: Handle Empty Result		If `validTours` is empty after filtering, the system prints `"No tours found."` and **returns immediately**, skipping the remaining steps.
Step 4: Sort by Total Amount		The `validTours` list is sorted in **descending order** based on the **Total Booking Amount**, calculated as `price × numTourists` for each tour. The sorting uses a custom comparator via `Double.compare(amount2, amount1)` to achieve descending order.

4.5 Function 5: Create a new booking

```
ALGORITHM Add New Booking BEGIN

  PRINT "--- Add new Booking ---"
  SET today = current date

  REPEAT
     INPUT bookingID (format: BXXXXX)
     Search booking record by bookingID

      IF booking already exists THEN
         PRINT "Error: Booking ID already exists."
      ELSE
         BREAK LOOP
      END IF
```

```
END REPEAT

INPUT fullName

REPEAT
    INPUT tourID (format: TXXXXX)
    Search tour record by tourID

    IF tour is NOT found OR tour is already booked THEN
        PRINT "Error: Tour not found or already booked."
    ELSE IF tour.departureDate is before today THEN
        PRINT "Error: Can not book tour has already started."
    ELSE
        BREAK LOOP
    END IF
END REPEAT

REPEAT
    INPUT bookingDate (format: dd/MM/yyyy)

    IF bookingDate is before today THEN
        PRINT "Error: Can not book tour in the past."
    ELSE IF bookingDate is after tour.departureDate THEN
        PRINT "Error: Can not book tour after departure date."
    ELSE
        BREAK LOOP
    END IF
END REPEAT

INPUT phone (10 digits)

Create a new Booking record with (bookingID, fullName, tourID, bookingDate, phone)
Add the new booking to the booking list
SET tour.booking = true

PRINT "Added booking: " + booking details + " successfully."

```

```
END
```

Listing 4.4: Create new booking

4.5.1 Booking Creation Workflow

Step	Description
Step 1: Booking ID Validation	The system prompts the user to input a Booking ID in `BXXXXX` format, validated by Regex (`Validator.BOOKING_ID_VALID`). It calls searchById(bookingID) to check for duplicates. If the ID already exists, an error is shown and the user must re-enter. The loop repeats until a unique ID is provided.
Step 2: Customer Name Input	The user enters their **Full Name** as a non-empty string. The input is collected using `Inputter.getString()` which prevents blank entries.
Step 3: Tour Association & Availability Check	The system prompts for a Tour ID in `TXXXXX` format. It queries `tourList.searchById(tourID)` and validates two conditions: (1) the tour must exist **and** not already be booked (`tour.isBooking() == false`), (2) the tour's departure date must not be before today. If any condition fails, an error is displayed and the user must retry.
Step 4: Booking Date Validation	The user inputs the Booking Date in `dd/MM/yyyy` format. The system validates: (1) the booking date must not be **before today** (no past bookings), and (2) the booking date must not be **after** the tour's departure date (must book before the tour departs). The loop repeats until a valid date is entered.
Step 5: Phone Input & Persistence	The user enters a **Phone Number** validated by Regex to ensure it is exactly 10 digits (`Validator.PHONE_VALID`). A new `Booking` object is created with the collected data and added to the booking list. The associated tour's `booking` status is set to `true` to mark it as booked. A success message is printed with the booking details.

4.6 Function 6: Remove booking

```
ALGORITHM Remove Booking BEGIN

  PRINT "--- Remove Booking ---"

  REPEAT
     INPUT bookingID (format: BXXXXX)
     Search booking record by bookingID

     IF booking is NOT found THEN
        PRINT "Error: This booking does not exist!"
```

```
        ELSE
            Find tour associated with booking.tourID

            IF tour.departureDate is before today THEN
                PRINT "Error: Can not remove tour has already started."
            ELSE
                BREAK LOOP
            END IF
        END IF
    END REPEAT

    Find tour associated with booking.tourID

    IF tour is NOT null THEN
        SET tour.booking = false
    END IF

    Remove booking from the booking list

    PRINT "Booking " + booking details + " has been removed successfully."

END
```

Listing 4.6: Remove Booking

4.6.1 Booking Cancelation Workflow

Step	Description
Step 1: Booking ID Validation	The system prompts the user to input a Booking ID in `BXXXXX` format, validated by Regex (`Validator.BOOKING_ID_VALID`). It calls searchById(bookingID) to check for duplicates. If the ID already exists, an error is shown and the user must re-enter. The loop repeats until a unique ID is provided.
Step 2: Customer Name Input	The user enters their **Full Name** as a non-empty string. The input is collected using `Inputter.getString()` which prevents blank entries.
Step 3: Tour Association & Availability Check	The system prompts for a Tour ID in `TXXXXX` format. It queries `tourList.searchById(tourID)` and validates two conditions: (1) the tour must exist **and** not already be booked (`tour.isBooking() == false`), (2) the tour's departure date must not be before today. If any condition

	fails, an error is displayed and the user must retry.
Step 4: Booking Date Validation	The user inputs the Booking Date in `dd/MM/yyyy` format. The system validates: (1) the booking date must not be **before today** (no past bookings), and (2) the booking date must not be **after** the tour's departure date (must book before the tour departs). The loop repeats until a valid date is entered.
Step 5: Phone Input & Persistence	The user enters a **Phone Number** validated by Regex to ensure it is exactly 10 digits (`Validator.PHONE_VALID`). A new Booking object is created with the collected data and added to the booking list. The associated tour's `booking` status is set to `true` to mark it as booked. A success message is printed with the booking details.

4.7 Function 7: Update booking

```
ALGORITHM Update Booking BEGIN

  PRINT "--- Update Booking ---"
  SET today = current date

  REPEAT
     INPUT bookingID (format: BXXXXX)
     Search booking record by bookingID

      IF booking is NOT found THEN
         PRINT "Error: This Booking does not exist!"
      ELSE
         BREAK LOOP
      END IF
   END REPEAT

   PRINT "Current Full Name: " + booking.fullName
   INPUT newFullName (Enter to skip)
   IF newFullName is NOT empty THEN
      SET booking.fullName = newFullName
   END IF

   INPUT newPhone (Enter to skip)
   IF newPhone is NOT empty THEN
      SET booking.phone = newPhone
   END IF

   REPEAT
      INPUT newTourID (Enter to skip)

      IF newTourID is empty OR newTourID equals booking.tourID THEN
         BREAK LOOP
      END IF

```

```
        Search new tour by newTourID

        IF newTour is NOT found OR newTour is already booked OR newTour.departureDate is before today THEN
            PRINT "Error: Tour not found, or already booked, or already started."
        ELSE
            Find oldTour by booking.tourID
            IF oldTour is NOT null THEN
                SET oldTour.booking = false
            END IF
            SET booking.tourID = newTourID
            SET newTour.booking = true
            BREAK LOOP
        END IF
    END REPEAT

    REPEAT
        INPUT newBookingDate (Enter to skip)

        IF newBookingDate is null THEN
            SET newBookingDate = booking.bookingDate
        END IF

        Find currentTour by booking.tourID

        IF newBookingDate is before today OR newBookingDate is after or equal to currentTour.departureDate THEN
            PRINT "Error: Booking date must be before tour departure date and >= today."
        ELSE
            SET booking.bookingDate = newBookingDate
            BREAK LOOP
        END IF
    END REPEAT

    PRINT "Booking " + booking details + " has been updated successfully."

END
```

Listing 4.7: Update booking

4.7.1 Booking Modification Workflow

Step	Description
Step 1: Booking Identification	The system prompts the user to input a Booking ID in `BXXXXX` format, validated by Regex (`Validator.BOOKING_ID_VALID`). It calls searchById(bookingID) to find the booking. If the booking does not exist, an error is shown and the user must re-enter. The loop repeats until a valid booking is found.
Step 2: Update Customer Info	The system displays the current Full Name and prompts for a new value. If the user presses Enter, the field is skipped. Similarly, the user can update the Phone Number (validated by `Validator.PHONE_VALID`). Each field is only updated if a

	non-empty value is entered.
Step 3: Update Tour Association	The user can enter a new Tour ID or press Enter to skip. If a new ID is entered, the system validates: (1) the new tour must exist, (2) it must not already be booked, and (3) its departure date must not be in the past. If valid, the **old tour's** `booking` status is set to `false` (releasing it), the booking's tour ID is updated, and the **new tour's** `booking` status is set to `true`.
Step 4: Update Booking Date	The user inputs a new Booking Date or presses Enter to keep the current value. The system retrieves the currently associated tour and validates: (1) the booking date must be **on or after today**, and (2) the booking date must be **before** the tour's departure date. The loop repeats until a valid date is provided.
Step 5: Persist & Confirm	After all fields have been processed, the system prints a success message displaying the updated booking details. All changes are applied in-memory and will be persisted when saveToFile() is called.

4.8 Function 8: Search Booking

```
ALGORITHM Search Bookings By Name BEGIN

    PRINT "--- Search Bookings ---"

    INPUT name

    PRINT table header "| BookingID | Full Name | Tour ID | Booking Date | Phone |"

    FOR EACH booking IN bookingList DO
        Convert booking.fullName to lowercase
        Convert name to lowercase

        IF booking.fullName contains name THEN
            PRINT booking.bookingID, booking.fullName, booking.tourID, booking.bookingDate (formatted), booking.phone
        END IF
    END FOR

END
```

Listing 4.8: Search Booking

4.8.1 Search Booking Workflow

Step	Description
Step 1: Input Search Keyword	The system prints the section header and prompts the user to input a name to search for. The input is a non-empty string collected via `Inputter.getString()`.
Step 2: Display Table Header	A formatted table header is printed showing the column names: Booking ID, Full Name, Tour ID, Booking Date, and

	Phone.
Step 3: Iterate and Match	The system iterates through every Booking in the booking list. For each booking, it performs a **case-insensitive partial match** by converting both `booking.getFullName()` and the search keyword to lowercase, then checking if the booking's full name **contains** the keyword using `String.contains()`. If matched, the booking's details are printed in a formatted table row using `printf`.

4.9 Function 9: StatisticTotalTourists

```
ALGORITHM Statistics Tourists Per Homestay BEGIN

    PRINT "--- Statistics: Tourists per Homestay ---"

    CREATE empty map homestayTouristCount (Key: homeID, Value: integer)

    FOR EACH homestay IN homestayList DO
        PUT (homestay.homeID, 0) INTO homestayTouristCount
    END FOR

    FOR EACH booking IN bookingList DO
        Find tour by booking.tourID

        IF tour is NOT null THEN
            GET current count from homestayTouristCount by tour.homeID (default 0)
            SET new count = current count + tour.numTourists
            PUT (tour.homeID, new count) INTO homestayTouristCount
        END IF
    END FOR

    PRINT table header "| Homestay Name | Tourists |"

    FOR EACH homestay IN homestayList DO
        GET count from homestayTouristCount by homestay.homeID
        PRINT homestay.homeName, count
    END FOR

END
```

Listing 4.9: StatisticTotalTourists

4.9.1 Statistics Workflow

Step	Description
Step 1: Initialization	The system prints the section header. A `HashMap<String, Integer>` called `homestayTouristCount` is created to store the total number of tourists per homestay. The map is initialized by iterating through all homestays and setting each homestay's count to `0`.
Step 2: Accumulate Tourist Counts	The system iterates through every `Booking` in the booking list. For each booking, it retrieves the associated `Tour` using `tourList.searchById(booking.getTourID())`. If the tour exists, the system gets the current tourist count for the tour's homestay from the map (defaulting to `0`), adds the tour's `numTourists` value, and updates the map entry. This accumulates the total tourists across all bookings for each homestay.
Step 3: Display Results	A formatted table header is printed. The system iterates through every `Homestay` in the homestay list, retrieves the accumulated tourist count from the map by `homeID`, and prints each homestay's name alongside its total tourist count in a formatted table row.

Chapter 6

6.1 Business Logic Architecture

6.1.1 Tour Management Logic

The system follows these rules for managing tours:

- Unique ID Validation: Every tour must have a unique ID following the format TXXXXX. The system validates the format using Regex and checks for duplicates by calling searchById(). If the ID already exists, the system displays an error and prompts the user to re-enter.

- Linked to a Homestay: A tour must be associated with a valid homestay. The system verifies the entered Home ID (format HSXXXX) against the homestay list using homestayList.searchById(). If the homestay does not exist, the tour cannot be created.

- Date Validation and Overlap Check: The user inputs both the Departure Date and End Date. The system enforces three rules: (1) the End Date must not be before the Departure Date, (2) the Departure Date must not be in the past, and (3) there must be no scheduling conflict with other tours using the same homestay. The overlap check is performed by isOverlap(), which ensures no two tours at the same homestay have overlapping date ranges.

- Capacity Constraint: The number of tourists must be strictly less than the homestay's maximum capacity. The system retrieves the maximumCapacity from the associated homestay and rejects any value that exceeds it.

- No Changes to Started Tours: To protect data integrity, the system prevents editing any tour whose Departure Date is before today. This ensures that tours that have already begun cannot be modified.

- Cascade Booking Deletion: When a tour's booking status is changed to false during an update, the system automatically removes all associated bookings from the booking list to maintain data consistency.

6.1.2 Booking Management Logic

The booking system uses the following rules to handle reservations:

- Tour Availability Check: Before creating a booking, the system verifies that the selected tour exists and is not already booked (isBooking == false). Additionally, the tour's Departure Date must not be in the past. If any condition fails, the booking is rejected.

- Valid Booking Date: A booking can only be created if the Booking Date meets two conditions: (1) it must be on or after today (no past bookings), and (2) it must be before the tour's Departure Date. This ensures bookings are only made for future, upcoming tours.

- Auto-updating Tour Status: The system automatically manages the tour's booking status. When a new booking is created, the associated tour is marked as booked (setBooking(true)). When a booking is removed, the system sets the tour's status back to available (setBooking(false)), allowing new bookings to be made.

- Safe Tour Reassignment: When updating a booking's Tour ID, the system performs a safe swap: the old tour's booking status is set to false (releasing it), and the new tour's status is set to true. The new tour must exist, must not already be booked, and must not have started. This prevents double-booking and data inconsistencies.

- Protection Against Removing Started Bookings: A booking cannot be removed if its associated tour has already started (Departure Date is before today), ensuring historical booking records are preserved.

6.1.3 Reporting and Analytics

The system provides reports to help managers make decisions:

- Filtering by Time: The system can filter tours into "Past" (departure date before today) and display them in a formatted table showing Tour ID, Tour Name, and Departure Date.

- Revenue-based Sorting: For upcoming tours (departure date after today) that have been booked, the system calculates the Total Booking Amount (Price x Number of Tourists) and sorts them in descending order. This allows managers to quickly identify the most valuable upcoming tours.

- Flexible Search: Users can search for bookings by entering a part of the customer's name. The search is case-insensitive and supports partial matching, so typing "binh" will find "Nguyen Van Binh" or "BINH TRAN".

- Tourists per Homestay Statistics: The system generates a summary report showing the total number of tourists for each homestay. It aggregates data from all bookings by mapping each booking to its tour's homestay, then accumulating the tourist counts. The result is displayed as a table with homestay names and their total tourist numbers.

Chapter 7

Conclusion

The Homestay Booking Management application was successfully developed using Java. The system implements ten core functions covering three main areas: Tour Management (add, update, list past tours, list upcoming tours sorted by revenue), Booking Management (add, update, remove, search by name), and Reporting (tourists per homestay statistics). Data persistence is achieved through text file I/O, ensuring that all tours, homestays, and bookings are saved and loaded reliably across sessions.